

1) La complejidad temporal de la inserción en el peor de los casos es **$O(N)$** . Esto ocurre cuando el árbol pierde su balance por completo (por ejemplo, al intentar insertar datos que ya vienen ordenados) y se "degenera" tomando la forma visual de una lista enlazada. En este escenario, el algoritmo se ve obligado a recorrer secuencialmente los N nodos existentes para poder agregar el elemento nuevo en el extremo final.

2)

1)

- A. El método main ejecuta un bucle que intenta insertar 50.000 números enteros de forma consecutiva y ascendente en el Árbol Binario de Búsqueda, llevando un contador para registrar cuántos elementos logró agregar antes de que el programa falle.
- B. La problemática evidenciada es que, al ingresar valores ya ordenados, el árbol pierde su balance y se "degenera" hasta convertirse funcionalmente en una lista enlazada. Esto expone la gran fragilidad de la implementación recursiva, que exige cada vez más recursos al recorrer linealmente la estructura, frente a la seguridad del enfoque iterativo.
- C. El StackOverflowError ocurre debido a que cada inserción recursiva en este árbol degenerado suma un nuevo marco de ejecución en la pila de llamadas (Call Stack) del sistema. Al anidarse miles de llamadas de forma continua, se supera rápidamente el límite de memoria estricto que la JVM le asigna a esa pila, provocando el colapso del programa.

3. Al comparar las implementaciones en un ABB, la **recursiva** ofrece un código más limpio, legible y fiel a la estructura del árbol, pero tiene la gran desventaja de consumir memoria de la pila por cada llamada ($O(N)$ en el peor caso). Esto la hace muy vulnerable a sufrir un StackOverflowError si el árbol se desbalancea o maneja grandes volúmenes de datos ordenados.

Por el contrario, la **iterativa** es mucho más segura y escalable para entornos productivos porque utiliza memoria constante ($O(1)$) al usar bucles y punteros locales, eliminando por completo el riesgo de desbordar la pila. Su única contrapartida es que el código se vuelve más largo, complejo de leer y propenso a errores lógicos durante el desarrollo.

3) El peor caso es cuando el árbol está completamente desbalanceado (un árbol degenerado). En este caso cada nodo sólo tiene hijo derecho, y buscar el último elemento requiere recorrer todos los nodos, por lo que su complejidad temporal es $O(n)$

Búsqueda binaria en Array:

Requiere que el array esté ordenado. Trabaja dividiendo el espacio de búsqueda a la mitad en cada paso.

Garantiza $O(\log n)$ siempre, aunque mantener el array ordenado después de inserciones y eliminaciones es caro ($O(n)$)

Búsqueda en ABB

Funciona similar a la búsqueda binaria, pero el orden no se impone sino que es propio de la estructura. La diferencia respecto a la búsqueda binaria es que la eficiencia depende del balance del árbol:

..Si el árbol está balanceado, la altura es $\log n$ y la búsqueda es $O(\log n)$.

..Si el árbol está degenerado, se convierte en una lista enlazada y la búsqueda es $O(n)$, igual que la búsqueda lineal.

5) La particularidad del recorrido **InOrden** es que imprime los valores ordenados de menor a mayor. Su complejidad temporal es **$O(N)$** porque obligatoriamente debe visitar todos los nodos del árbol.